FPT Education
FPT UNIVERSITY

Chapter 10: MTD Subsystem

FPT Education
FPT UNIVERSITY

Learning Objectives

- Understand the Memory Technology Device (MTD) subsystem and its role in abstracting raw Flash memory complexities
- Learn to configure MTD services, mapping drivers, and specific Flash chip drivers in the Linux kernel
- Master methods for defining MTD partitions using Redboot tables, kernel command lines, and mapping drivers
- Utilize MTD user space utilities to erase, manage, and program Flash memory images effectively
- Build and deploy Flash-optimized file systems (JFFS2 and UBIFS) with wear-leveling as root file systems

FPT Education
FPT UNIVERSITY

Memory Technology Device

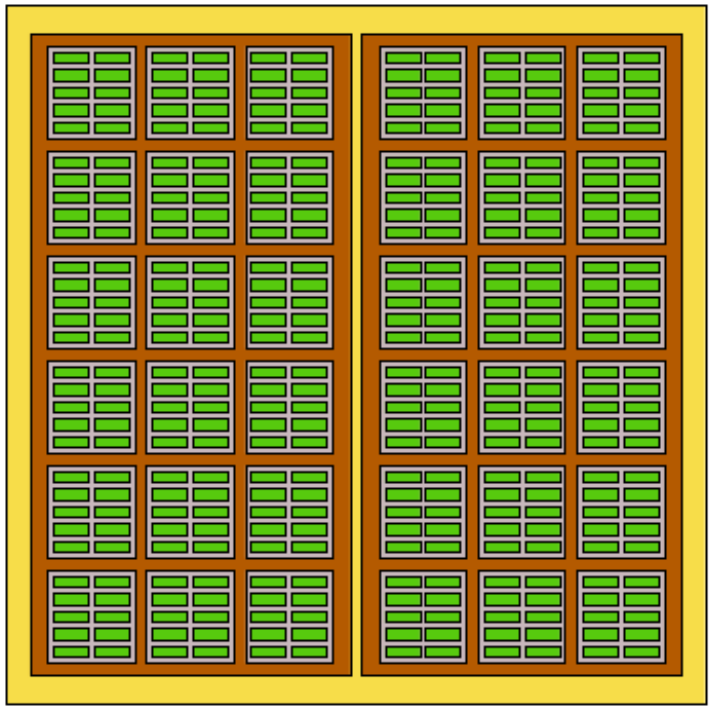
MTD Overview

hệ thống con của Linux dùng để làm việc với bộ nhớ Flash thô (raw Flash) như NOR và NAND.

- Raw Flash Access (MTD)
  - MTD provides direct access to Flash memory without block device abstraction, exposing low-level hardware behavior.
  - NOR Flash: NOR Flash được ánh xạ vào không gian địa chỉ của CPU; CPU đọc lệnh tại các địa chỉ đó như đọc bộ nhớ.
    - Byte-addressable read (random access)
    - Supports XIP (Execute In Place) XIP (Execute In Place) trên NOR Flash là khả năng CPU thực thi chương trình trực tiếp từ Flash, không cần chép chương trình sang RAM trước.
    - Fast read, slow write/erase, lower density
  - NAND Flash:
    - Page-based read (e.g., 2KB–16KB)
    - No XIP (must load to RAM)
    - High density, fast write, needs ECC + bad block handling

ECC – Error Correction Code – là mã giúp phát hiện và sửa lỗi bit trong giới hạn cho phép.

- Erase-before-Write Model
  - Data must be erased in large blocks before writing in smaller pages, unlike traditional disks.
- Bad Blocks Handling
  - Flash devices contain defective blocks that must be detected and avoided during operation.





Operation	Area
Read	Page
Program (Write)	Page
Erase	**Block**

https://flashdba.com/2014/06/20/understanding-flash-blocks-pages-and-program-erases/

MTD Partitions

chia một chip Flash thô thành nhiều vùng để lưu bootloader, kernel và root filesystem.

- Logical Partitions
  - Flash memory is divided into logical regions (bootloader, kernel, root filesystem) for structured storage and management.
- Device Tree (Modern)
  - Partition layout is defined in the Device Tree, allowing flexible and hardware-independent configuration.
- Kernel Command Line (Legacy) khai báo qua tham số khởi động
  - Partitions can be specified using mtdparts= parameter, mainly for debugging or older systems.

FPT Education

FPT UNIVERSITY

DTS Partitions

cách U-Boot truyền cho Linux hai thông tin khi khởi động:
cách chia NAND Flash và phân vùng nào chứa root filesystem.

Device Tree

```
flash@0 {
   compatible = "jedec,spi-nor";   // hoặc NAND controller
node
   reg = <0x0 0x2000000>;          // tổng dung lượng flash (ví
dụ 32MB)

   partitions {  Các phân vùng có vị trí và kích thước cố định
      compatible = "fixed-partitions";
      #address-cells = <1>;  Offset được biểu diễn bằng một số 32 bit
      #size-cells = <1>;  Kích thước được biểu diễn bằng một số 32 bit

      bootloader@0 {
         label = "bootloader"; Tên phân vùng
         reg = <0x00000000 0x00100000>;  // 1MB
         read-only;  chỉ đọc
      };

      kernel@100000 {
         label = "kernel";
         reg = <0x00100000 0x00400000>;  // 4MB
      };

      rootfs@500000 {
         label = "rootfs";
         reg = <0x00500000 0x01B00000>;  // phần còn lại
      };
   };
};
```

Vị trí bắt đầu và kích thước, tính bằng byte

Tham số yêu cầu Linux phân tích layout phân vùng MTD

- Legacy Bootarg – mtdparts= (Debug/Bring-up)

Thiết bị Flash cần chia

Phân vùng tên bootloader, kích thước 1 MiB

mtdparts=nand0:1m(bootloader),4m(kernel),-(rootfs)

Phân vùng tên rootfs, chiếm toàn bộ phần còn lại

  - Cú pháp: <device>:<size>(name),...
  - '-' = dùng toàn bộ dung lượng còn lại
  - Áp dụng qua bootargs (U-Boot / kernel cmdline)

- Using mtdparts in U-Boot:

```
setenv bootargs console=ttyS0,115200 \
mtdparts=nand0:1m(bootloader),4m(kernel),-(rootfs) \
root=/dev/mtdblock2 rootfstype=jffs2
boot
```

FPT Education

FPT UNIVERSITY

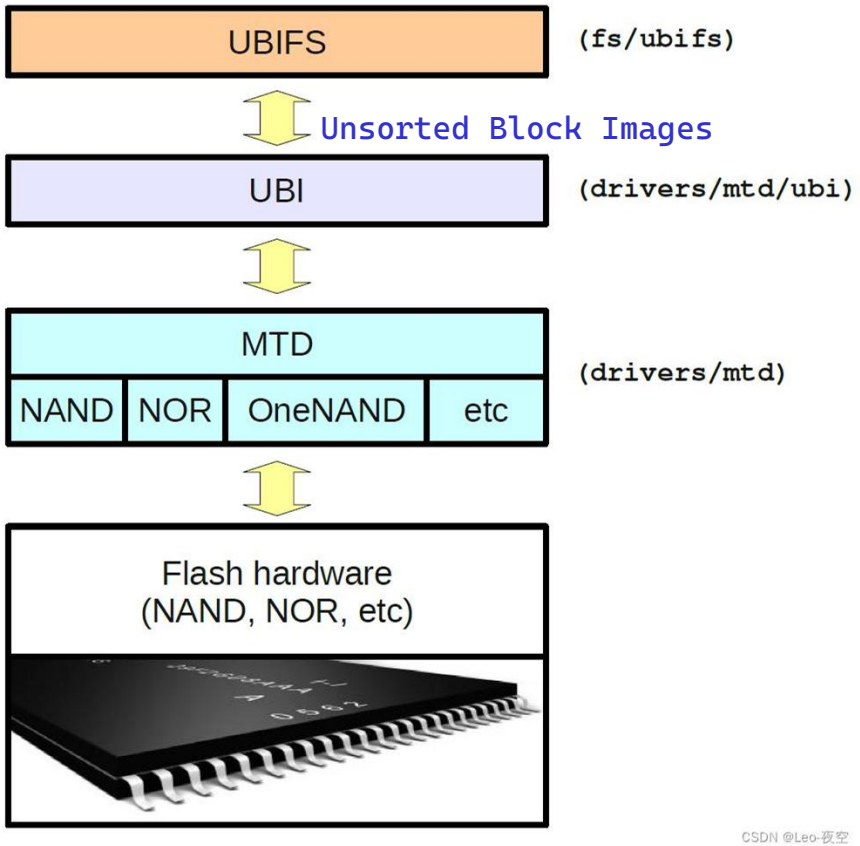
Drivers & Initialization

```
Flash 128 MB
├── 1 MB   bootloader   /dev/mtd0
├── 8 MB   kernel       /dev/mtsd1
└── 119 MB rootfs       /dev/mtd2
```

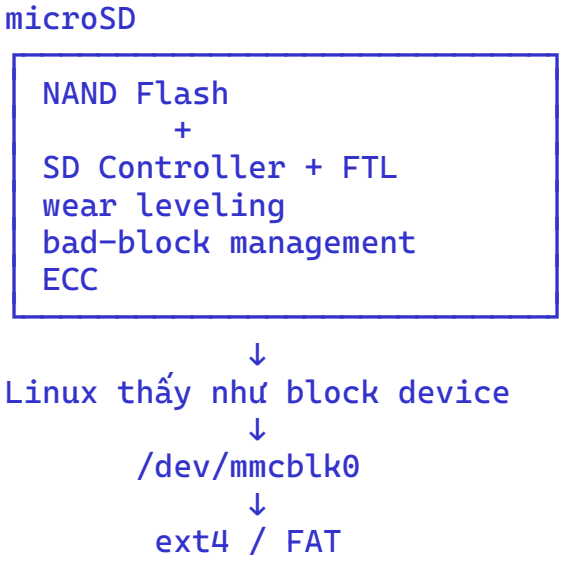
- Mapping drivers define layout
- NAND/NOR drivers handle hardware
- Board init binds Flash to MTD

UBI – Unsorted Block Images (drivers/mtd/ubi)
- Nằm trên MTD.
- Quản lý các erase block của Flash.
- Hỗ trợ bad block management, wear leveling và volume management.

filesystem chạy trên UBI.
- Kernel nhìn thấy file/thư mục bình thường như /etc, /bin, /home, ...



CSDN @Leo-夜空


microSD
NAND Flash
+
SD Controller + FTL
wear leveling
bad-block management
ECC
Linux thấy như block device
/dev/mmcblk0
ext4 / FAT

FPT Education

FPT UNIVERSITY

MTD Utilities & JFFS2

các công cụ thao tác trực tiếp lên phân vùng Flash thô

- MTD Utilities
  - Tools like flash_erase, flashcp, and nandwrite are used to erase, program, and manage raw Flash partitions.

Erase entire partition (ALL blocks)

Phân vùng cần xóa

flash_erase /dev/mtd2 0 0 Xóa các khối trong phân vùng Flash

Bắt đầu từ offset 0 của phân vùng

Xóa đến cuối phân vùng

Ghi file vào Flash, thường dùng cho NOR; thực hiện xóa, ghi và kiểm tra

-p Padding: bổ sung dữ liệu để trang cuối đủ kích thước

Write image to NAND (handle bad blocks, padding)

Phân vùng đích File image cần ghi

nandwrite -p /dev/mtd2 kernel.bin

Ghi image vào NAND, có xử lý đặc tính trang và bad block

OOB – Out-of-Band là vùng phụ đi kèm mỗi trang NAND, có thể chứa mã ECC, dấu bad block và thông tin quản lý khác, tùy thiết kế.

Đọc nội dung NAND ra file

Read raw NAND to file (with OOB/ECC options as needed)

Phân vùng nguồn

nanddump /dev/mtd2 -o -f dump.bin

-o: Đọc kèm vùng OOB Lưu dữ liệu đọc được vào file dump.bin

VD: NAND có trang 2048 byte, file dài 5000 byte:
5000 = 2*2048 + 904
Hai trang đầu đầy đủ; trang cuối chỉ có 904 byte.
Với -p, công cụ bổ sung:
2048 – 904 = 1144 byte

Show flash geometry (erase size, page size, OOB)

mtdinfo /dev/mtd2

FPT Education
FPT UNIVERSITY

UBI & UBIFS

- UBI (Unsorted Block Images)
  - Manages raw Flash by handling wear leveling and bad block management across NAND devices.
- UBIFS (UBI File System)
  - A Flash-aware filesystem running on top of UBI, providing efficient storage and fast access.
- Scalability
  - Designed for large NAND Flash, offering faster mount times and better performance than JFFS2.

FPT Education
FPT UNIVERSITY

cách tạo, mount và sử dụng UBIFS trên NAND Flash.

Using UBIFS

- Building UBIFS Image
  - Use mkfs.ubifs to create filesystem image and ubinize to package it into a UBI volume.

tạo filesystem UBIFS từ thư mục root filesystem.

đóng gói UBIFS thành UBI volume/image để ghi lên NAND.

- Mounting UBIFS
  - Mounted by the kernel using parameters like rootfstype=ubifs and root=ubi0:rootfs.

tên UBI volume chứa root filesystem.

root filesystem dùng UBIFS.

thiết bị UBI số 0

- Deployment
  - UBIFS is widely used as the standard root filesystem for NAND Flash-based embedded systems.

kết hợp với UBI để xử lý tốt các đặc điểm của NAND như:
- bad block
- wear leveling
- erase block
- lỗi/mất điện tốt hơn so với filesystem thông thường.

FPT Education
FPT UNIVERSITY

Using UBIFS (2)

mtdinfo cho biết các thông tin như:
Minimum I/O unit size – kích thước page nhỏ nhất có thể đọc/ghi.
Eraseblock size – kích thước block xóa vật lý.
Số erase block.
Kích thước tổng của partition.

- Building UBIFS Image
  - # 1) Check NAND/MTD geometry first
    - mtdinfo /dev/mtd2
  - # 2) Create UBIFS image with required geometry
    - mkfs.ubifs -r rootfs/ \ lấy nội dung thư mục rootfs/ để tạo filesystem

      -m 2048 \ minimum I/O size = 2048 byte, thường tương ứng NAND page = 2 KiB

      -e 126976 \ kích thước LEB – Logical Erase Block của UBI

      -c 2000 \ số LEB tối đa filesystem có thể sử dụng

      -o rootfs.ubifs file UBIFS được tạo

kiến trúc thực tế:

UBIFS
↓
UBI
↓
MTD
↓
NAND

đóng gói UBIFS vào một UBI volume:
rootfs/
↓
mkfs.ubifs
↓
rootfs.ubifs
↓
ubinize
↓
ubi.img

ubi.img mới chứa cấu trúc UBI volume,
chẳng hạn volume có tên: rootfs

FPT Education
FPT UNIVERSITY

Using UBIFS (2)

Ghi UBI image xuống NAND

1) Flash UBI image to NAND (MTD) → writes UBI image to raw flash

hoặc: ubiformat /dev/mtd2 -f ubi.img

```
nandwrite -p /dev/mtd2 ubi.img
```

sau khi ghi:
/dev/mtd2
└── chứa UBI image

2) Attach MTD → UBI layer (REQUIRED) → exposes MTD as UBI device (/dev/ubi0)

attach MTD device số 2 vào UBI.

```
ubiattach /dev/ubi_ctrl -m 2
# (creates /dev/ubi0, /dev/ubi0_*)
```

/dev/mtd2
↓
ubiattach
↓
/dev/ubi0

sau khi attach:
/dev/ubi0: UBI device
/dev/ubi0_0: volume số 0
/dev/ubi0_1: volume số 1

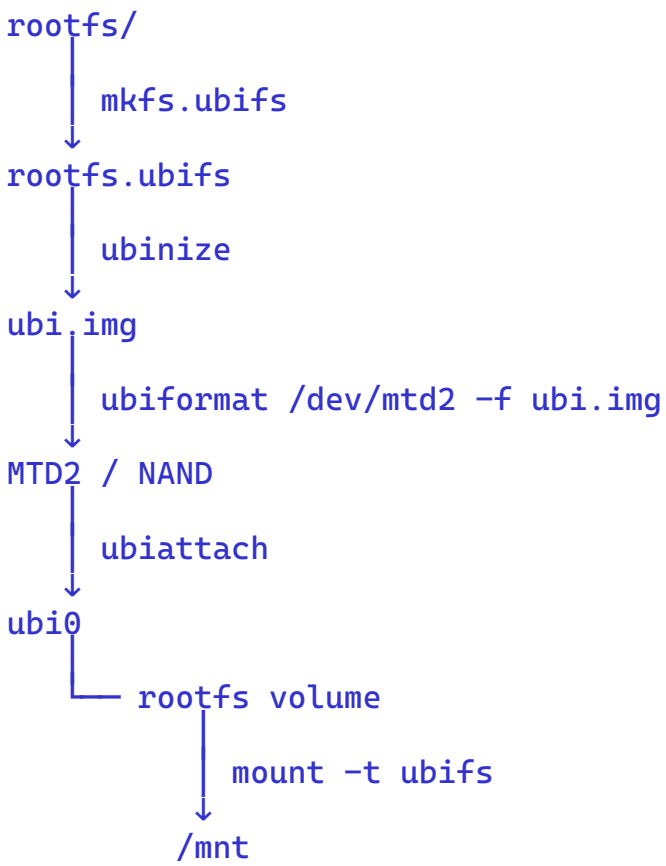
3) Mount UBIFS volume → mounts a volume (e.g., ubi0:rootfs)

filesystem type là UBIFS.

thư mục mount point.

```
mount -t ubifs ubi0:rootfs /mnt
```

volume tên rootfs của UBI device ubi0.


rootfs/
mkfs.ubifs
rootfs.ubifs
ubinize
ubi.img
ubiformat /dev/mtd2 -f ubi.img
MTD2 / NAND
ubiattach
ubi0
rootfs volume
mount -t ubifs
/mnt

RedBoot Flash Partition Tables (Legacy)

- RedBoot is an older bootloader (eCos-based)
- Uses Flash Image System (FIS) to define partitions
- Partition table stored directly in flash

Name	Flash Addr	Size
bootloader	0x00000000	0x00100000
kernel	0x00100000	0x00400000
rootfs	0x00500000	...

Trong các hệ embedded Linux hiện đại, partition Flash thường được mô tả bằng:
- Device Tree
- fixed partitions
- đôi khi mtdparts= trong kernel command line cho debug/bring-up.

Summary

- MTD Subsystem: Provides essential support for memory devices, isolating low-level Flash complexities from the kernel
- Configuration: Requires enabling MTD, selecting specific Flash chip drivers, and defining hardware mapping
- Partitioning: Flash memory can be managed as a single device or divided using Redboot, command-lines, or mapping drivers
- MTD Utilities: User space tools are critical for safely managing, erasing, and flashing images onto MTD devices
- File Systems: JFFS2 and its successor, UBIFS, are robust, Flash-optimized file systems that provide crucial wear-leveling features